PAPER PRESENTATION · ASPLOS 2025

# ReCA:

# Integrated Acceleration for Real-Time and Efficient Cooperative Embodied Autonomous Agents


**PAPER AUTHORS**

Zishen Wan, Yuhang Du, Mohamed Ibrahim, Jiayi Qian, Jason Jabbour, Yang (Katie) Zhao, Tushar Krishna, Arijit Raychowdhury, and Vijay Janapa Reddi


---

**COMS 6998 — AI-Native Computing**

Columbia University

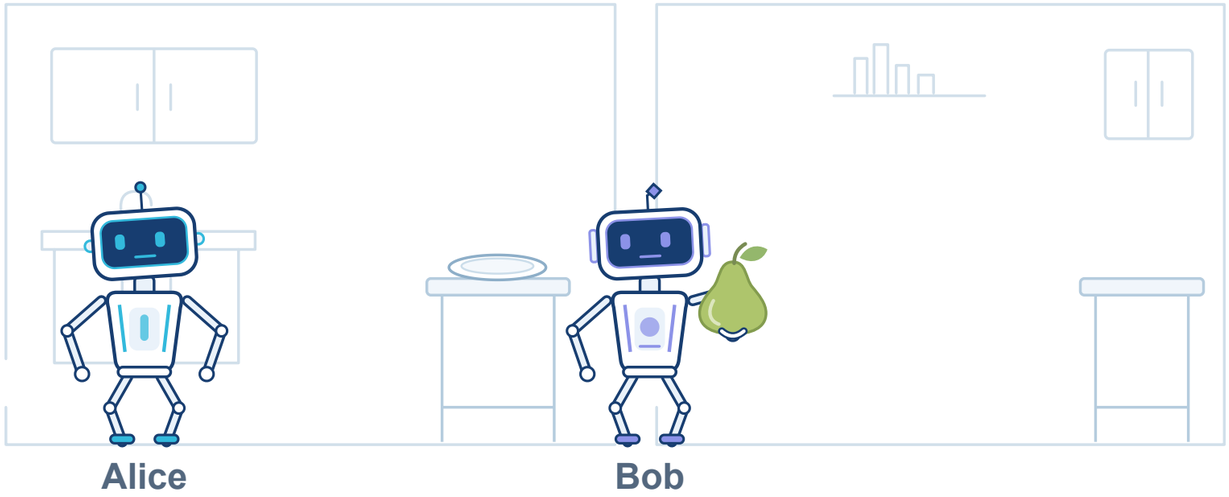

**PRESENTED BY**

**Yishan Song & Shurui Hu**

# How Do Two Agents Coordinate One Mission?

ANIMATION EXAMPLE

## Alice and Bob: Fetching One Pear




Adapted from ReCA Fig. 2

MISSION GOAL

## Prepare One Plate

1 Pear + 2 Apples + 1 Pudding

CURRENT SUBTASK

## Bob Retrieves the Pear

Alice remembers that Bob previously spotted pears and asks him to retrieve one while she continues exploring.

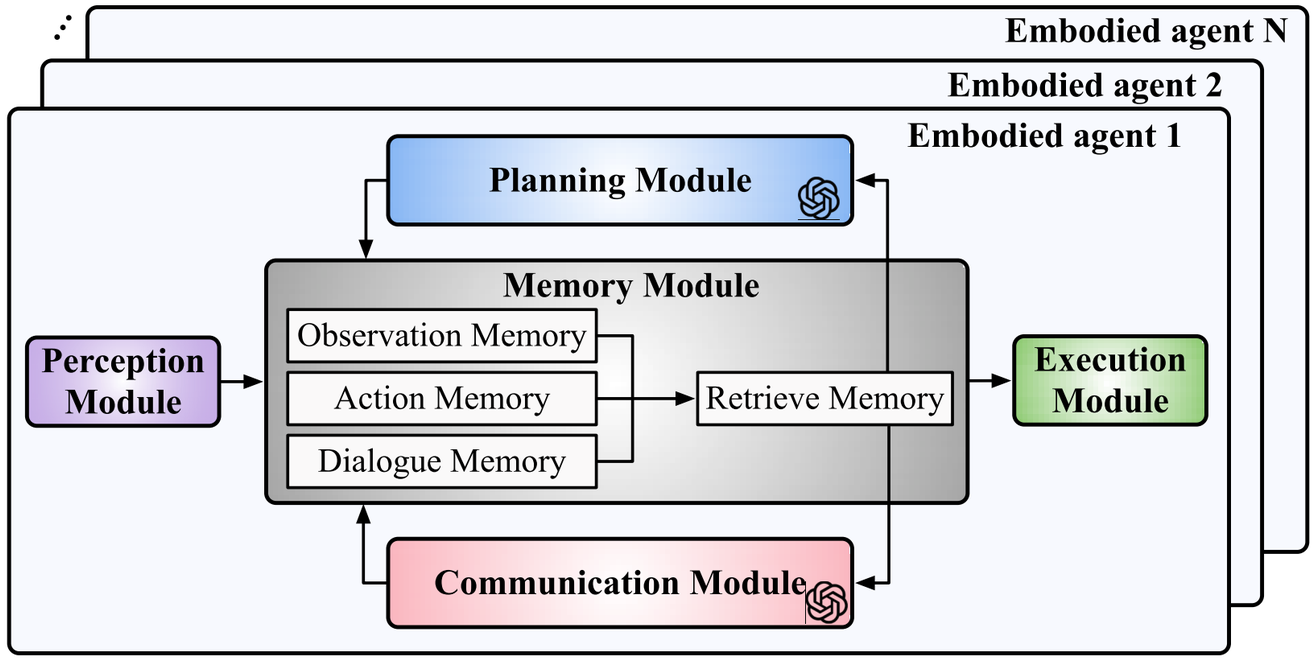
COOPERATION FLOW

**Recall → Request → Retrieve**

ONE TAKEAWAY

**Shared information enables coordinated action.**

# Inside the Embodied Agent Workflow




ReCA, Fig. 1 · Modular Framework for Cooperative Embodied AI Agents

RESEARCH QUESTION

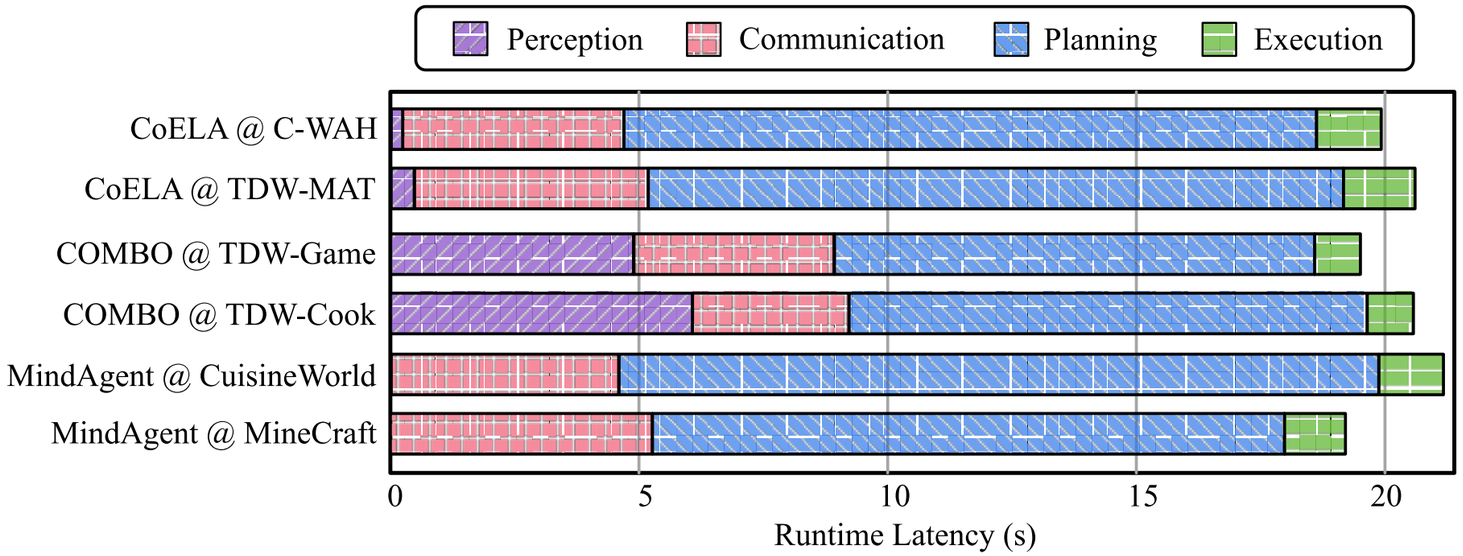
## What happens inside Bob when Alice asks him to fetch a pear?

KEY INSIGHT

**An embodied agent is a workflow, not just an LLM.**

Cooperation connects multiple agent workflows.

# Where Does the Time Go? - Bottleneck Analysis




ReCA, Fig. 3 / §§3.1–3.2 · Per-Step Latency Profiling

**RESEARCH QUESTION**

**Why can a single step take so long for agents like Bob?**

**PROFILING RESULT**

**~20 s / step**

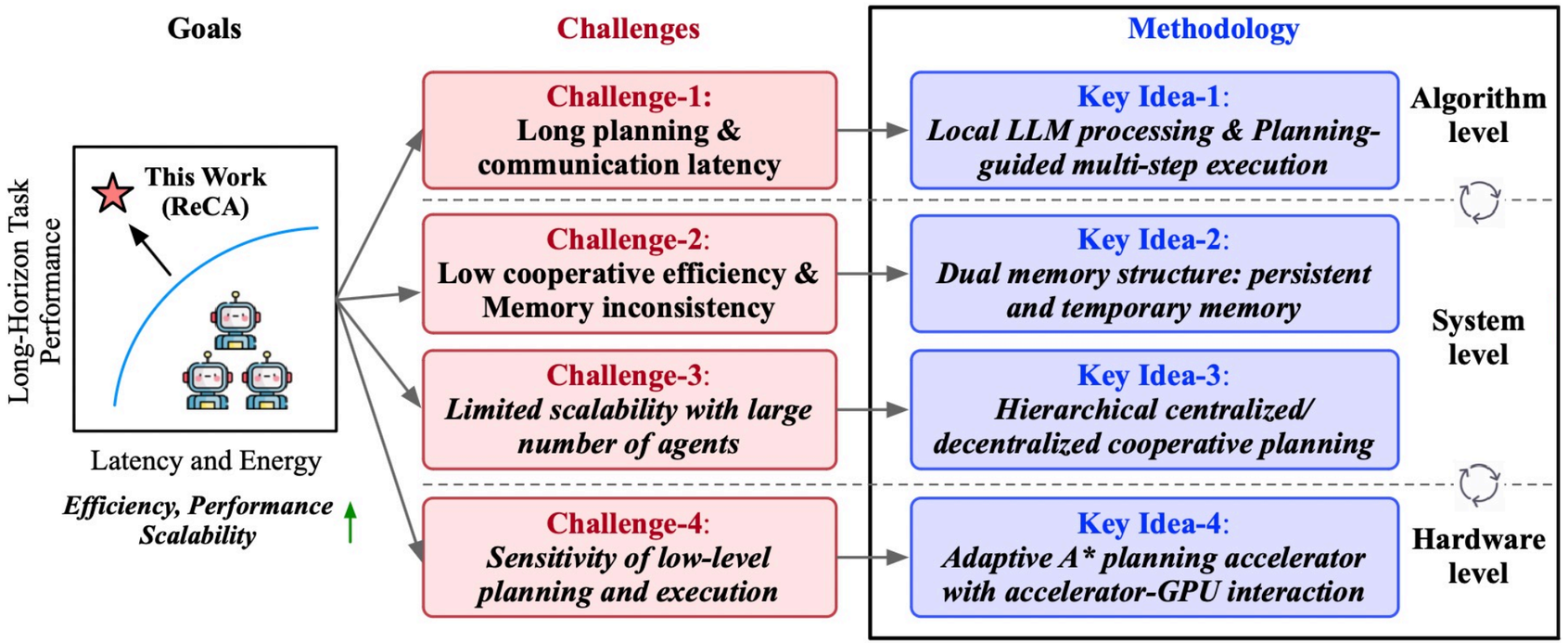
End-to-end processing time per profiled agent step in CoELA, COMBO, and MindAgent—not total mission time.

**KEY FINDING**

**LLM-based planning and communication dominate runtime.**

# ReCA: Optimize the Entire Mission

**FULL OVERVIEW** Algorithm, system, and hardware work together




**RESEARCH QUESTION**

How can Bob and the team **finish faster?**

**RECA'S APPROACH**

Match each bottleneck to a **cross-layer solution.**

**KEY INSIGHT**

Optimize the **entire mission**, not just one LLM call.

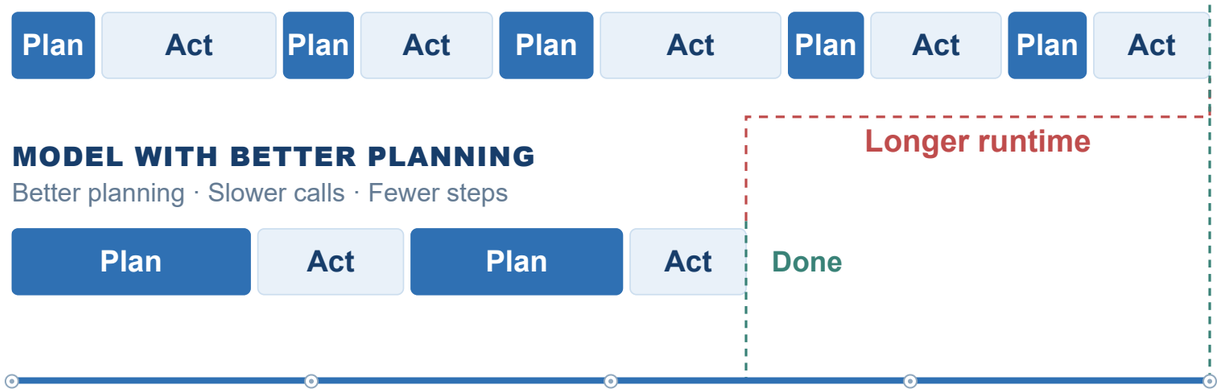
Zishen Wan, ASPLOS 2025 presentation, Slide 35 · ReCA §§3.4, 4, 6.2.

# Is a Smaller LLM Always Faster at Completing Tasks?

ANIMATION EXAMPLE

## Bob's task: fetch a pear.






ReCA'S APPROACH

## Optimize Both Sides

### Planning Quality

Reduce unnecessary task steps

- LoRA
- CoT Prompting
- Hierarchical Prompts

### Inference Efficiency

Reduce execution overhead

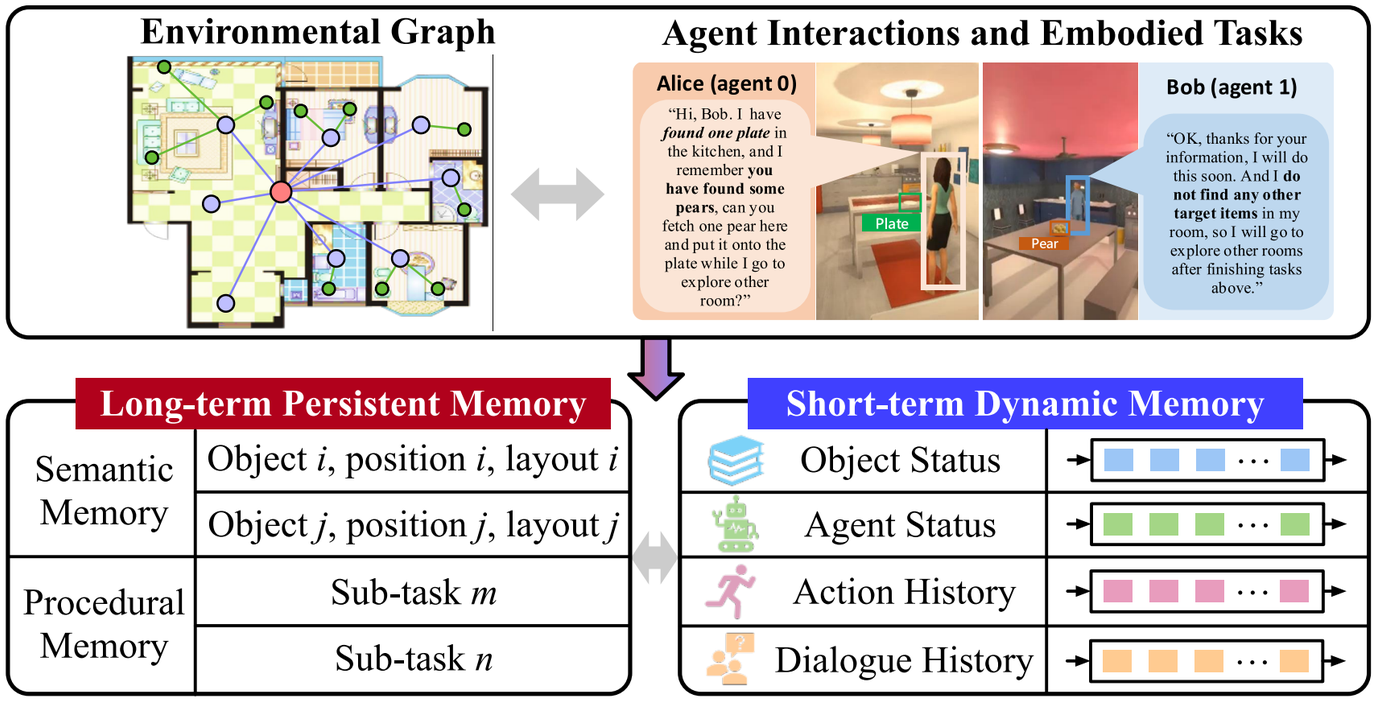
- MLC-LLM
- AWQ Quantization

TAKEAWAY

**Faster Inference ≠ Faster Mission**

ReCA §5.1, p. 987.

# What Should Agents Remember?




## Dual Memory Structure

### LONG-TERM PERSISTENT MENORY

Expand the **hierarchical graph** as new areas are explored; serialize it into **structured text** for the LLM planner.

### SHORT-TERM / DYNAMIC

Replace **least recently used data** with recent information; prioritize **task-relevant details** to prevent **context dilution**.

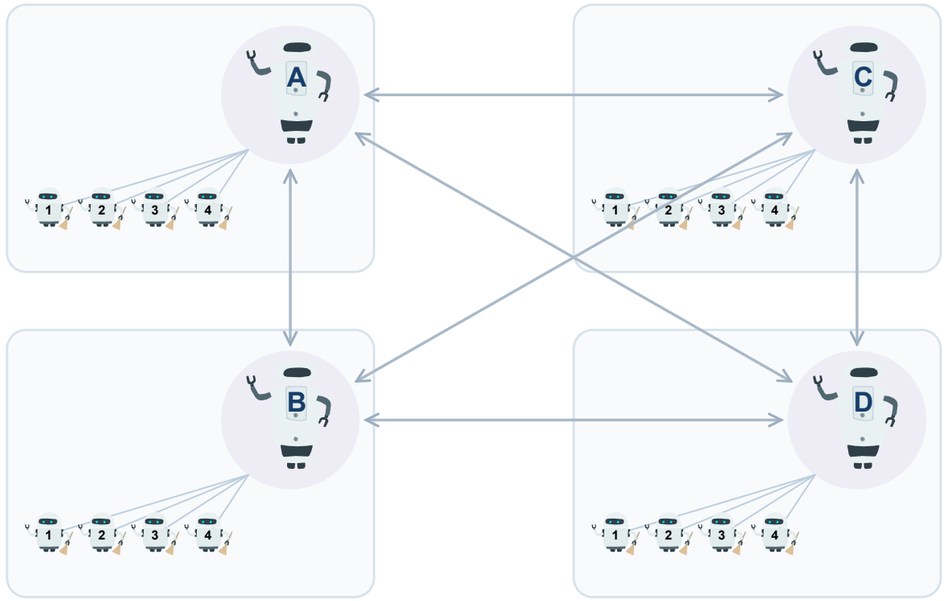
### TAKEAWAY

**Minimize redundant exploration.**

ReCA §5.2 · Fig. 7, p. 988

# Hierarchical Planning for Scalable Cooperation

## ReCA's
### Hierarchical Cooperative Planning




THE SCALABILITY TRADE-OFF

### Decentralized

Each agent plans and communicates.

Repetitive dialogue → higher latency and context dilution.

### Centralized

One LLM planner coordinates the team.

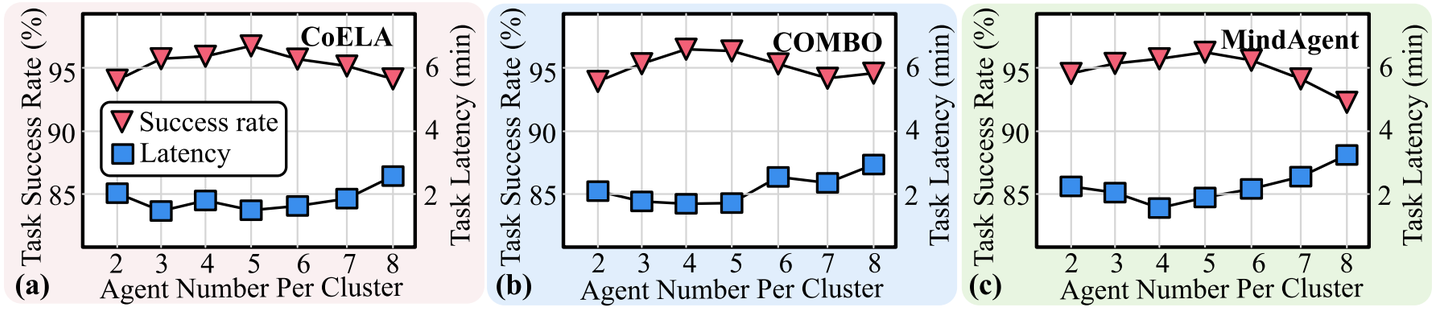
Larger teams → lower task success rates.

### Hierarchical

Cluster parents **distribute planning workload.**
Parents exchange progress across clusters,
reducing **communication overhead** and **context dilution.**

ReCA §3.3, p. 986

# How Large Should a Cluster Be?




**TAKEAWAY**

Too few or too many agents per cluster negatively impact **task success rates** and increase **end-to-end latency**, particularly in large-scale systems.

**ReCA · Best Overall Performance**

**5** agents per cluster

**SMALL CLUSTERS**

Having too few agents makes the system behave like a **decentralized architecture**.

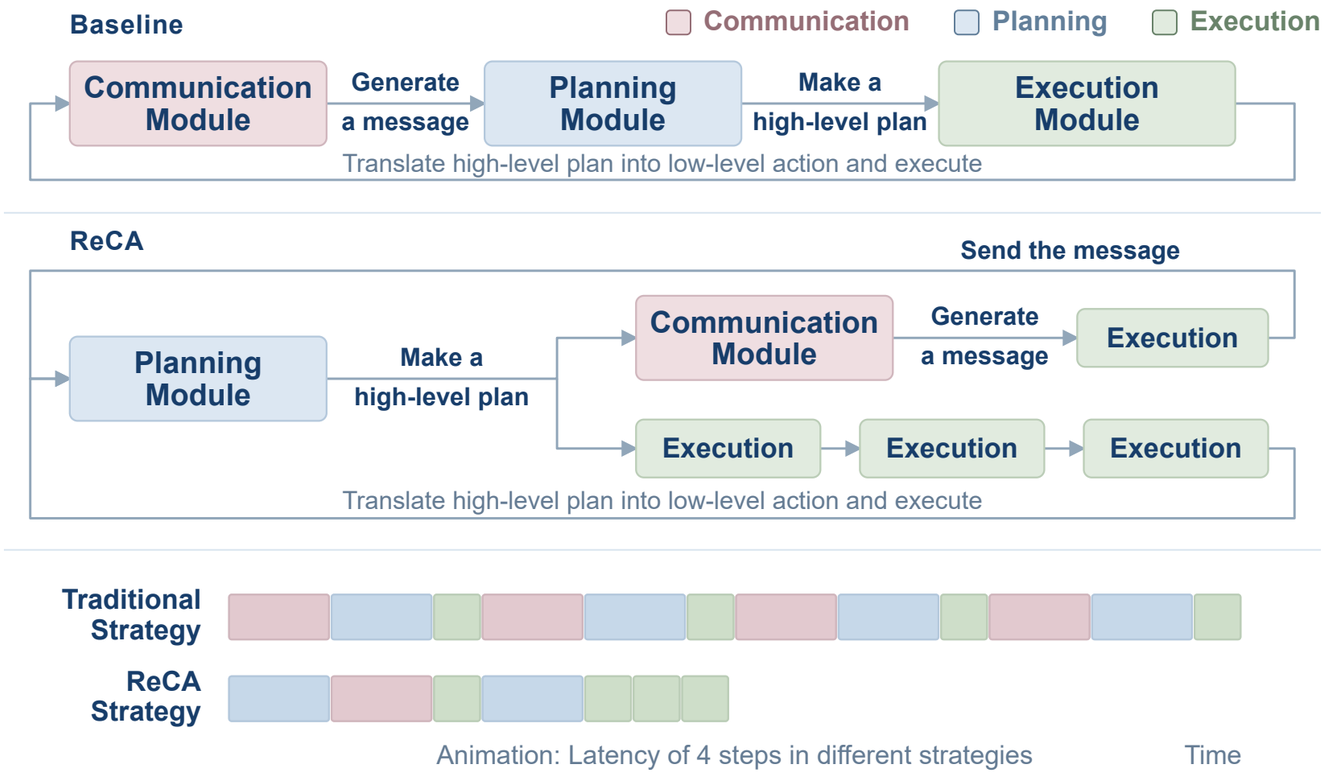
**LARGE CLUSTERS**

Having too many agents leads to **centralized behavior**.

ReCA §7.3 · Fig. 16, pp. 993–994

# Plan First. Communicate When Needed.

**RESEARCH QUESTION** Do Bob and Alice need to re-plan and generate a message at every step?




Animation: Latency of 4 steps in different strategies

**PLAN → COMMUNICATE**

## Generate messages when essential.

Assess communication needs using dual memory.

**PLAN → ACT → ACT**

## One high-level plan, multiple actions.

Guide consecutive low-level actions within a specified time frame.

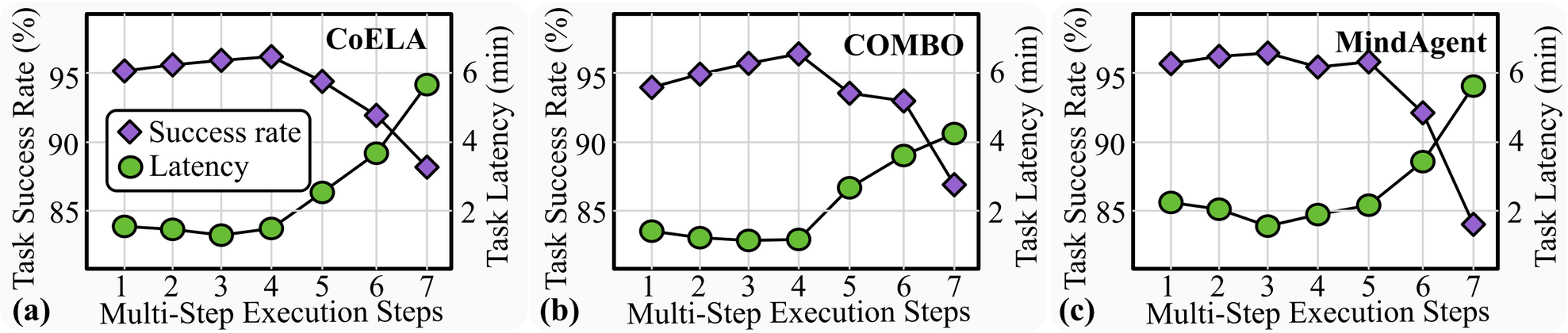
## Fewer LLM inferences.
## Lower end-to-end latency.

Self-reflection monitors agent statuses and triggers re-planning when necessary.

ReCA §5.4 · Fig. 9, p. 989.

# How Many Actions Should Follow One Plan?

**RESEARCH QUESTION** **Why can't Bob just keep going without re-planning?**




ReCA, Fig. 15 · Sensitivity to Multi-Step Execution Length

Fig. 15 measures execution-length sensitivity, not the isolated effect of self-reflection. The best horizon is workload-dependent.

**THE TRADE-OFF**

| | |
|---|---|
| **Too short** | Frequent LLM planning |
| **Too long** | Accumulated trajectory errors |

**RECA'S APPROACH**

| | |
|---|---|
| **Pre-deployment** | Sweep execution horizons |
| **Runtime** | Self-reflection & re-planning |

# The Bottleneck Has Moved

### Before Optimization




### After High-Level Optimization




Adapted from ReCA §6.2

**RESEARCH QUESTION**

**If Bob can think faster, what's slowing him down now?**

**BOTTLENECK SHIFT**

**5–15% → 20–40%**

A* share of total latency

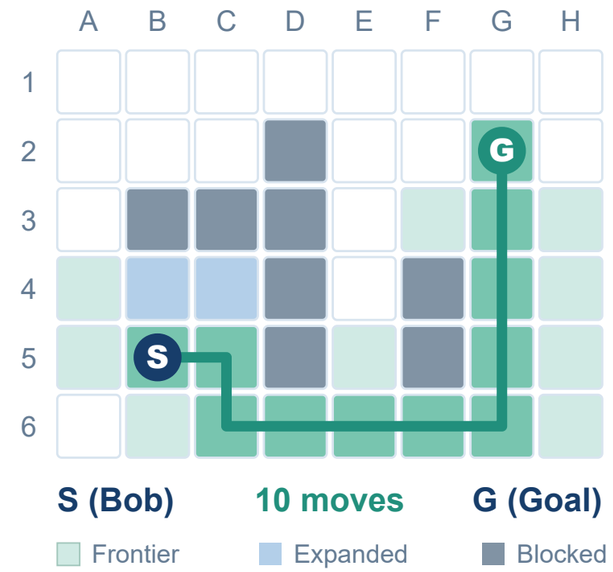
**KEY FINDING**

**Optimizing LLM processing reveals low-level A* planning as the next bottleneck.**

# Why Is A* Hard to Accelerate?

## A* Path Planning

4-way moves · Manhattan h




## f(n) = g(n) + h(n)

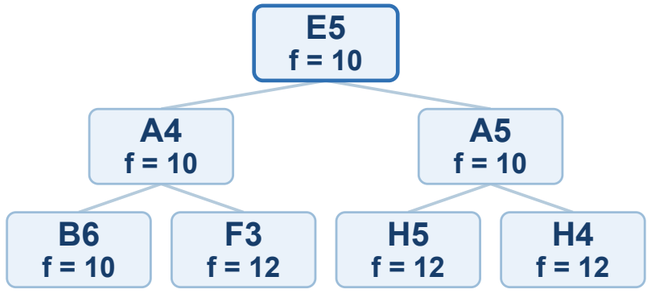
g: moves so far · h: estimated moves left

G2 · stored best route

**g = 10 · h = 0 → f = 10**

**Goal popped → reconstruct the stored route**

## Min Heap / Priority Queue

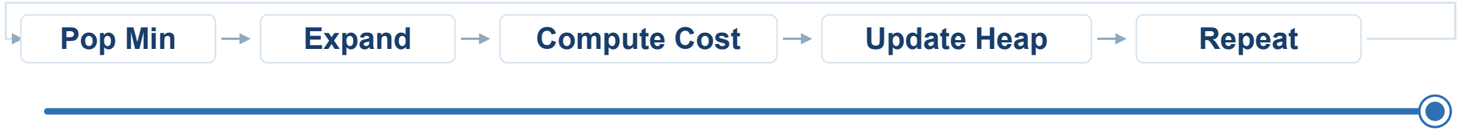



Top 7 of 9 frontier entries

**Stop after goal extraction**

**Goal popped → Path found**

Remaining frontier nodes are not expanded




Adapted from ReCA §§6.2–6.3

### RESEARCH QUESTION

**Why is Bob's A* search costly on general-purpose CPUs and GPUs?**

### COMPUTATIONAL CHALLENGES

**Sequential Dependencies**

Each expansion depends on the updated search state.

**Irregular Control and Memory Access**

Frequent heap updates and data-dependent branching.

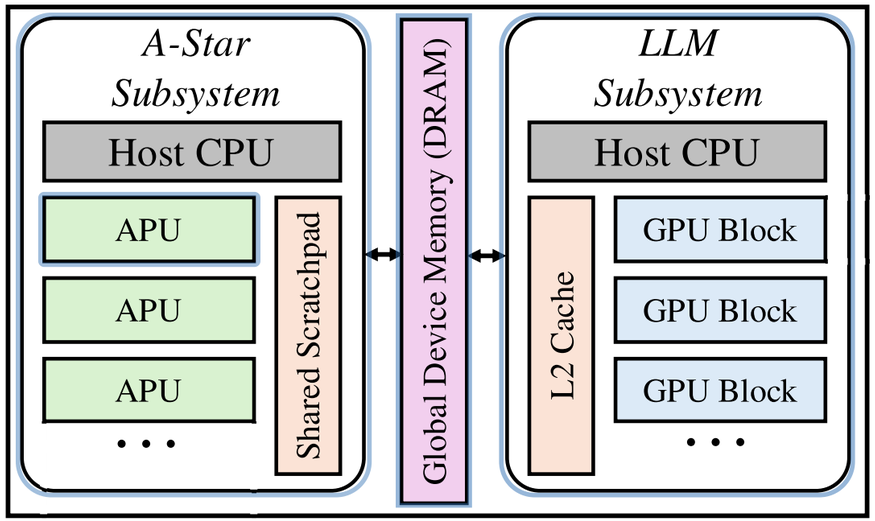
### KEY FINDING

**CPU: repeated heap and memory operations.**

**GPU: dependent iterations limit parallelism.**

# Matching LLMs and A* to the Right Hardware

## **GPU** for LLM inference · **APU** for A* search




ReCA, Fig. 10(a) · Integrated Hardware Architecture

**Shared DRAM** exchanges plans and search feedback.

**Shared scratchpad** keeps search state across iterations and links APUs with their host.

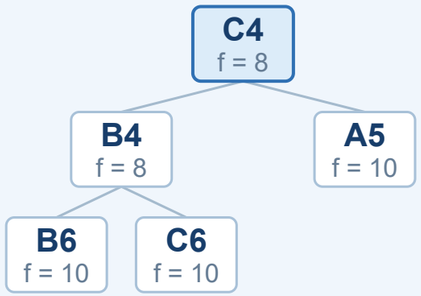
INSIDE ONE APU

## One A* Iteration

Unit-cost search
Current cell C5

**APU controller coordinates the iteration**

**Heap Unit (HU)**
Frontier min heap




**Local scratchpad**
Current node

**C5**
g = 1

Neighbor data

**C4**

**C6**

costs stored

**Cost Compute Unit**
(CCU)

**f(n) = g(n) + h(n)**

**Both neighbors in parallel**

**2 + 6 = 8**

**2 + 8 = 10**

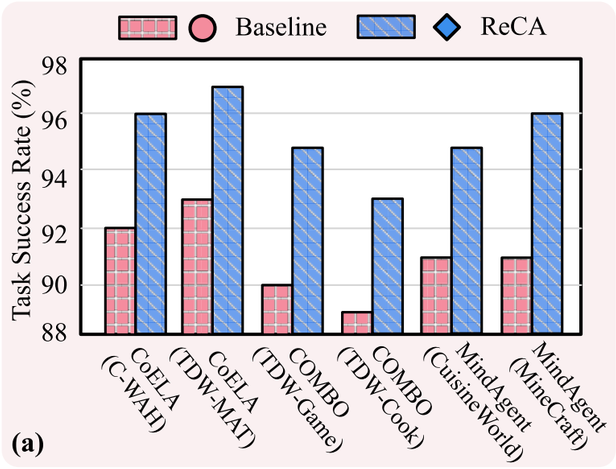
**Specialized hardware accelerates each A* iteration without removing sequential dependencies.**

ReCA, Fig. 10(a) · APU mechanism adapted from Figs. 10(b), 11(c), §§6.1–6.4 · ASPLOS '25

# Does ReCA Actually Speed Up the Mission?

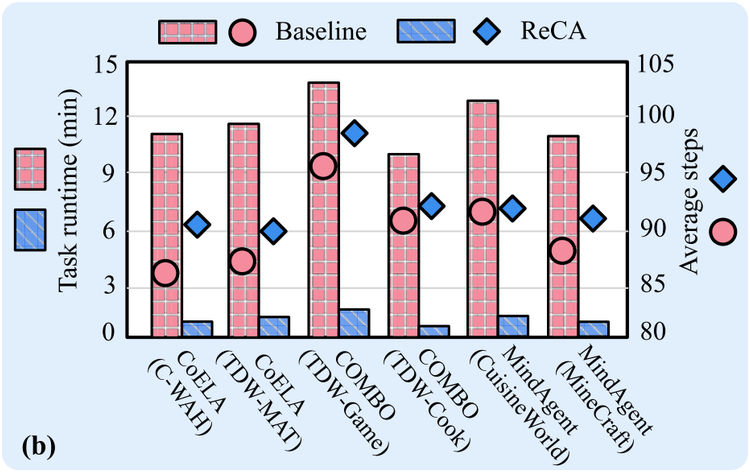
## Task success

Higher is better




## End-to-end runtime

Lower is better · dots show task steps




Six benchmarks, three systems, four agents. Runtime and steps are measured on successful missions.

**ReCA ACHIEVED**

**10.2×** average end-to-end speedup

and **4.3% improvement** in task success rate, with only **3.2% more steps**.

**TAKEAWAY**

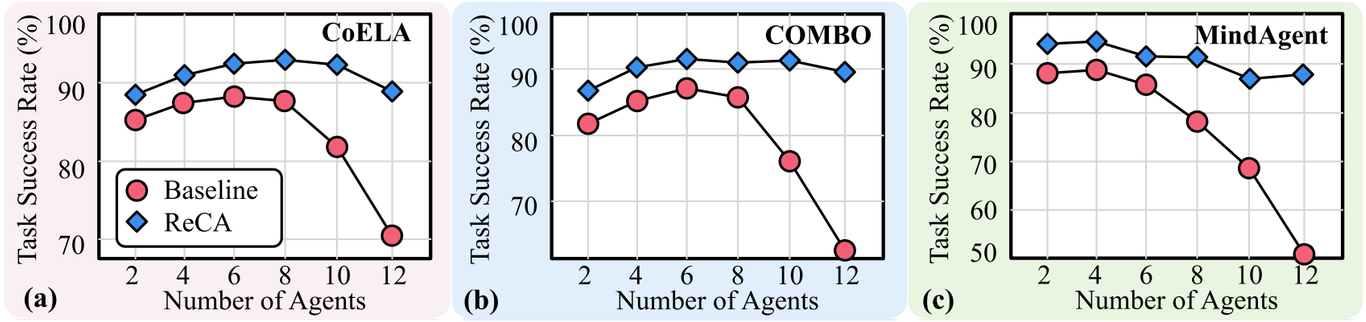
**The mission is faster because each step takes less time.**

ReCA, §7.1–7.2, Fig. 12, p992-993.

# Scalability with More Agents

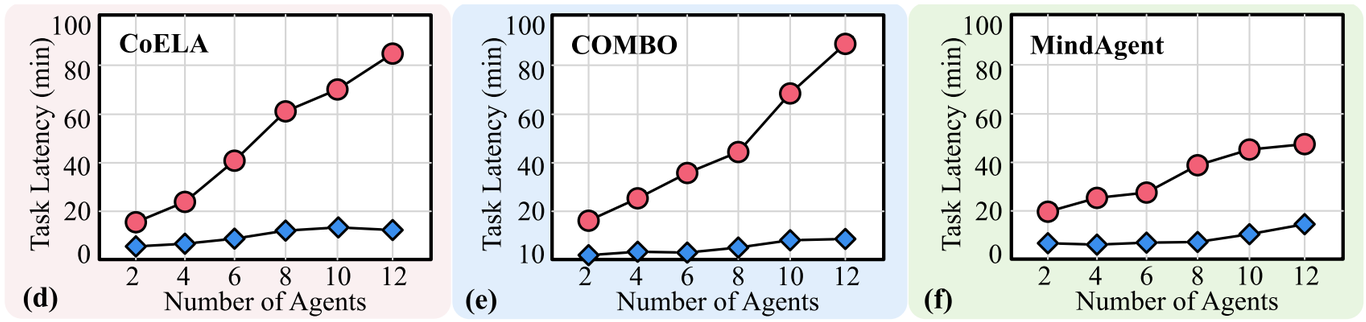
## Task Success Rate

Higher is better




## End-to-End Latency

Lower is better




AS THE NUMBER OF AGENTS INCREASES

**ReCA maintains higher task success rates and lower latency than the baselines.**

AT 12 AGENTS

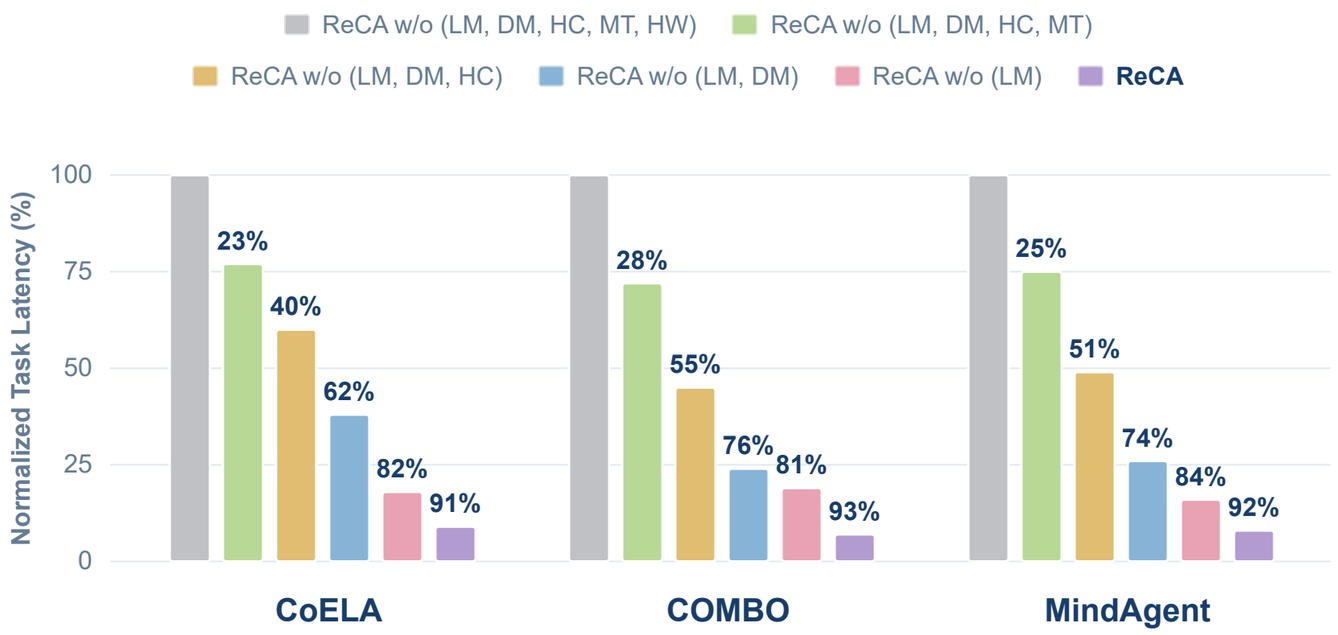
| Metric | Baseline | ReCA |
|---|---|---|
| Task success rate | <70% | ≈90% |
| End-to-end latency | ≈1 hour | ≤20 min |

TAKEAWAY

**ReCA maintains task performance and efficiency with more agents.**

ReCA §7.2, Fig. 13, p. 993.

# Are All ReCA Optimizations Effective?




Ablation study on ReCA optimization techniques.

## SUCCESSIVE CONFIGURATIONS

All five optimizations removed

\+

**HW:** A-star/GPU heterogenous hardware system — Hardware

\+

**MT:** multi-step execution — Software

\+

**HC:** hierarchical cooperation — Software

\+

**DM:** dual memory — Software

\+

**LM:** local model — Software

**= ReCA**

## TAKEAWAY

**Model-system-hardware co-design** is important for system performance.

ReCA §7.3 · Fig. 17, p. 994

# Conclusion & Related Work

## CONCLUSION

ReCA is an **integrated software-hardware co-design framework** for **efficient and scalable cooperative agent systems**.

## MAIN CONTRIBUTION

01 **Dual Memory Structure**

02 **Hierarchical Cooperative Planning**

03 **Planning-Guided Multi-Step Execution**

04 **Scalable A-Star Accelerator**

This paper proposed a new perspective on **scalable and efficient cooperative embodied agent systems**.

## RELATED IDEAS

**Arborist** Yaotian Liu et al. · 2026 · MICRO · co-author: Zishen Wan

Accelerates FMT* using parallel tree expansion and speculative look-ahead with in-order commit.

**FLASH** Jiahui Niu et al. · 2026 · arXiv

Uses a lightweight draft model, parallel verification, and phase-aware fallback to accelerate VLA inference.

**Deltoris** Zheng Liu et al. · 2026 · arXiv

Combines temporal-aware bit-level sparsity and speculative inference to reduce VLA compute and data traffic.

**REMAC** Haoxuan Wang et al. · 2026 · ICLR

Uses masked action chunking and prefix-preserved sampling for reliable asynchronous robot execution.

ReCA §9 · p. 994